EPI Educational Group

Maintenance prédictive & Anticipation des pannes disques durs

Architecture CNN-LSTM / GRU / TRANSFORMER

Backblaze

Réalisé par: Salma Mlika — SG02 · Mai 2026

Modèles Stochastiques – Projet

PLAN

01 Introduction & Problématique

02 Dataset Backblaze

03 Prétraitement des données

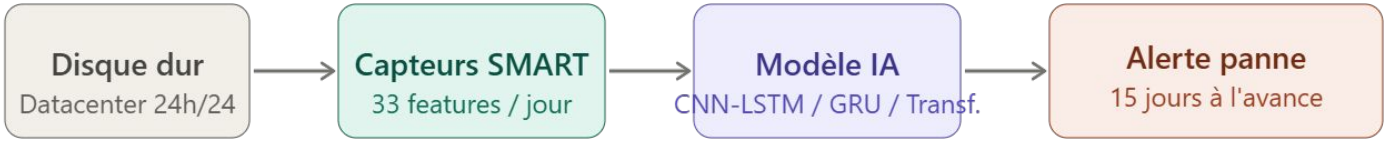
04 Construction des séquences

05 Architecture CNN-LSTM

06 Architecture GRU Transformer

07 Conclusion

Contexte & Problématique



Métrique prioritaire : **Recall**

FN = panne manquée = catastrophe industrielle

- **Datacenter : milliers de disques en fonctionnement 24h/24**
- **Panne non anticipée → perte données, interruption service, coûts élevés**

- **Objectif : prédire la panne 15 jours à l'avance via l'historique SMART**
- **Métrique prioritaire : Recall — un FN = panne manquée = catastrophe**

Dataset Backblaze

date	serial_number	failure	smart_*_raw
2017-01-03	Z305B2QN	0	0, 14, 0, ...
2017-01-03	PL1331LAGG	0	2, 0, 7, ...
2017-09-17	Z305XXLA	1 ← panne	88, 0, 341, ...

154 colonnes SMART brutes → suppression constantes → 33 features

- ➔ 1 ligne = 1 disque × 1 jour, relevé quotidien
- ➔ Train : 2017 (4 trimestres) / Test : Q4 2016
- ➔ **fortement déséquilibré** — pannes < 1% des observations
- ➔ 154 colonnes → **33 informatives** après suppression des constante

Déséquilibre des classes



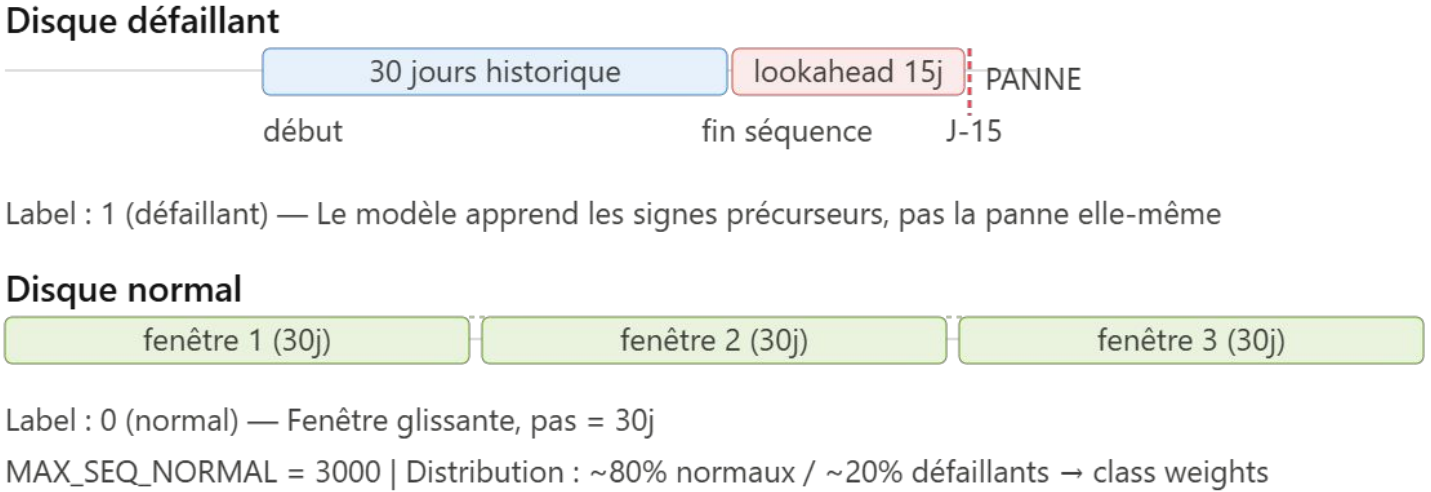
Prétraitement des données et Gestion du déséquilibre

Étape	Action	Résultat
1	CSV → Parquet par trimestre	RAM maîtrisée
2	Suppression colonnes constantes	154 → 33 features
3	Suppression colonnes `_normalized`	Garder uniquement `_raw`
4	Tri par `serial_number` + `date`	Ordre temporel garanti
5	Forward fill par disque	Jours manquants comblés
6	StandardScaler	Moyenne 0, écart-type 1

	Disques normaux	Disques défaillants	Ratio
Dataset brut	~69 000	~865	1 panne / 80 normaux
Après sélection	1 000	~865	~1 panne / 1 normal
Après séquences	~3 000 séq.	~696 séq.	1 panne / 5.7 normaux

Niveau	Solution	Effet
Sélection disques	Garder les 1000 normaux avec le plus de données	Réduit le déséquilibre brut
Construction séquences	MAX_SEQ_NORMAL = 3000	Limite l'explosion des normaux
Entraîneme nt	Class weights (≈ 0.62 / 2.50)	Pénalise plus les erreurs sur pannes

Construction des Séquences



- ➔ `SEQUENCE_LENGTH = 30j`, `LOOKAHEAD_DAYS = 15j`
- ➔ **Défaillantes** : 30j se terminant 15j **avant** la panne → apprendre les signes précurseurs
- ➔ **Normales** : fenêtre glissante (pas = 30j) sur disques sains
- ➔ Shape finale : `(3696, 30, 33)` — ~80% normaux / 20% défaillants
- ➔ Class weights pour compenser le déséquilibre résiduel

Architecture CNN-LSTM & Résultats

CNN : extrait les patterns locaux dans les signaux capteurs (variations rapides, anomalies ponctuelles)
LSTM : capture la dégradation progressive sur30 jours



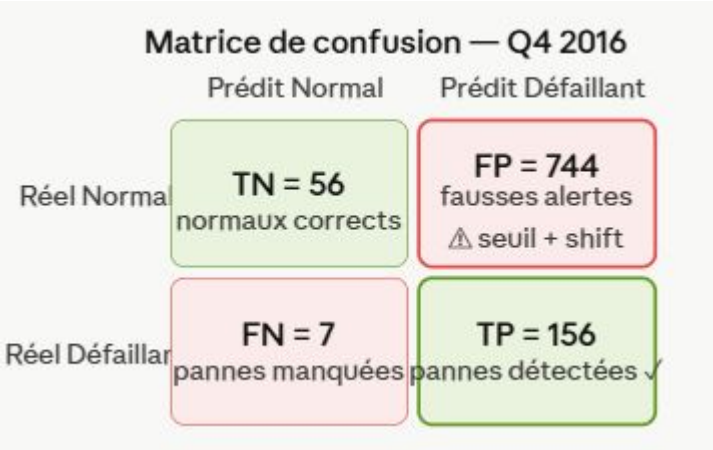
Résultats validation 2017

Train	Acc=1.00	Recall=0.98	AUC=0.99
Val	Acc=0.99	Recall=0.96	AUC=0.98

➔ Légère baisse train→val = overfitting léger, contrôlé par EarlyStopping
➔ 5-6 pannes manquées seulement sur la validation

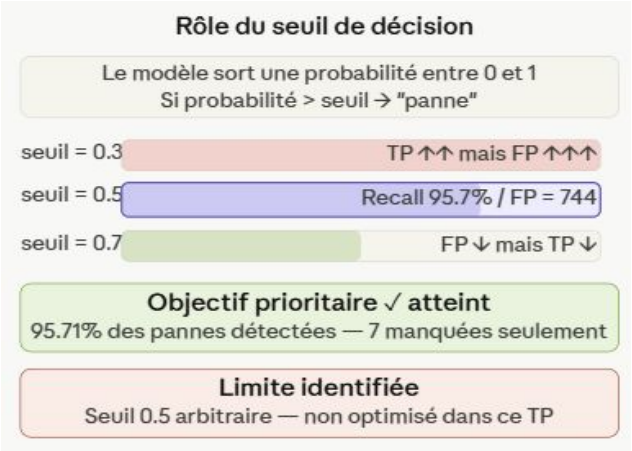
Test Généralisation Q4 2016 (CNN-LSTM)

Résultats clés :



- 156 pannes détectées sur 163 → **Recall = 95.71%**
  objectif principal atteint ✅
- 7 pannes manquées (FN) → très faible ✅
- 744 fausses alertes (FP) → problème de précision sur les normaux ⚠️

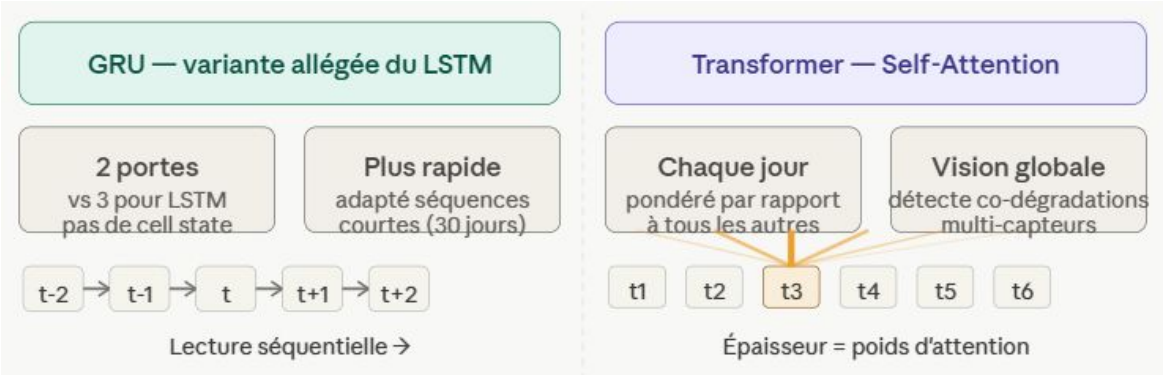
Pourquoi 744 fausses alertes ?



Deux causes combinées :

1. **Seuil = 0.5** (non optimisée) → Le modèle dit "panne" dès que probabilité > 50% → Choix non optimisée — un seuil à 0.6 ou 0.7 pourrait réduire les FP
2. **Distribution shift** (2016 ≠ 2017) →profils SMART 2016 ≠ 2017, scaler calibré sur 2017 → les normaux 2016 ressemblent à des défaillants 2017

Pourquoi Explorer d'Autres Architectures ?



- CNN-LSTM performant mais limites identifiées :
  - CNN : fenêtre locale fixe → peut rater des dépendances entre instants éloignés
  - LSTM : séquentiel, plus lourd à entraîner

Question : **quelle architecture capture le mieux la dynamique du processus temporel ?**

➔ Une panne = souvent la **conjonction de plusieurs capteurs dégradés simultanément** → nécessite une vision globale de la séquence

Deux alternatives :

- **GRU** : variante allégée du LSTM (2 portes vs 3, pas de cell state séparé) → plus rapide, adapté aux séquences courtes (30j)
- **Transformer** : Self-Attention → chaque jour est pondéré par rapport à **tous** les autres → détecte les co-dégradations multi-capteurs

Résultats GRU vs Transformer

Modèle	Accuracy	AUC	Recall	F1	Params
CNN-LSTM	0.9946	0.9820	0.9600	0.9800	~50k
GRU	0.9892	0.9965	0.9424	0.9704	29 505
Transformer	**0.9946**	**0.9976**	**0.9784**	**0.9855**	62 977

Analyse :

- ➔ **Transformer** : meilleur recall (97.84%) et AUC (0.9976) → mécanisme d'attention idéal pour détecter les co-dégradations
- ➔ **GRU** : recall plus faible (94.24%) mais 2× moins de paramètres → bon compromis si contrainte ressources
- ➔ **CNN-LSTM** : bon équilibre, architecture de référence

Recall = capacité du modèle à ne pas rater les pannes(TP)

AUC =capacité du modèle à distinguer les pannes des normaux.

Analyse Comparative & Conclusion

- ➔ **Recall prioritaire en maintenance prédictive → mieux vaut détecter un maximum de pannes → Transformer = meilleur choix**
- ➔ **GRU justifié : sur 30 jours, le modèle n'a pas besoin de mémoriser des informations très lointaines → architecture simple suffisante**
- ➔ **Transformer justifié : les signes d'une panne peuvent apparaître à différents moments de la séquence → l'attention repère les jours importants**
- ➔ **Distribution shift Q4 2016 → les données changent avec le temps → problème/performance indépendant du modèle choisi**

- ➔ **Trade-off final : Transformer (plus performant) vs GRU (plus léger) vs CNN-LSTM (bon équilibre)**



Merci pour votre attention

CREDITS: Diese Präsentationsvorlage wurde von **Slidesgo** erstellt, inklusive Icons von **Flaticon** und Infografiken & Bilder von **Freepik**